

All helpers return an object shaped like:

```
{ <mainResultField>: ..., error: ... }
```

Teammates MUST always check error before using the result.

Section 1 – Auth & Profiles

1. signUpWithEmail({ email, password, displayName })

What it does

- Creates a new user in **auth.users**.
- Creates/initializes a profile later via `getCurrentUserProfile` (not in this function itself).
- Sets `user_metadata.display_name` to the given `displayName`.

Tables / Systems touched

- `auth.users` (Supabase Auth)

Use when

- User signs up with email/password.

Watch out for

- Will fail (error non-null) if:
 - Email already in use.
 - Password too weak / invalid.
 - There was a server-side issue (status 500).
 - After signup, user is usually already logged in (session active).
-

2. signInWithEmail({ email, password })

What it does

- Logs in the user with email/password.

Tables / Systems touched

- `auth.users` (Supabase Auth)

Use when

- Existing user logs in.

Watch out for

- On wrong credentials, user will be null and error will describe the issue.
 - After success, you can call `getCurrentUser()` / DB helpers that rely on auth.
-

3. `signOut()`**What it does**

- Logs out the current user (invalidates session on client).

Tables / Systems touched

- Supabase Auth session (no DB tables directly).

Use when

- User presses “Log out”.

Watch out for

- After this, any helper that depends on auth (`likePost`, `createPost`, etc.) will fail with “No authenticated user” until user logs in again.
-

4. `getCurrentUser()`**What it does**

- Returns the current auth user object from Supabase.

Tables / Systems touched

- Supabase Auth; not a Postgres table query.

Use when

- You need user id (`user.id`) or email (`user.email`) in the UI.

Watch out for

- If the user is not logged in, user is null.

- Many helpers internally call this; you rarely need it directly unless building UI or debugging.
-

5. `getCurrentUserProfile()`

What it does

- Ensures there is a profile row for `auth.uid()`.
- If a profile row exists in **public.profiles**, returns it.
- If not, **inserts a new profile** row using `user.id` and `user_metadata.display_name`.

Tables / Systems touched

- `public.profiles`

Use when

- You need profile info (display name, avatar, bio).
- Good to call on app startup after login to ensure profile exists.

Watch out for

- Requires user to be logged in; otherwise returns error ("No authenticated user").
 - Depends on RLS policies in profiles:
 - `SELECT/INSERT/UPDATE` restricted to `auth.uid() = id`.
 - This function is also your “safety net” to create the profile row if something went wrong at signup time.
-

6. `updateCurrentUserProfile({ displayName, bio, avatarUrl })`

What it does

- Updates the current user’s row in **public.profiles**.

Tables / Systems touched

- `public.profiles`

Use when

- User edits profile in settings (name, bio, avatar URL).

Watch out for

- Requires user to be logged in.
 - RLS must allow UPDATE for auth.uid() = id.
 - If you pass undefined for a field, it usually won't be updated (depending on implementation); pass null explicitly to clear a value if needed.
-

Section 2 – Daily Prompts & Roulette

Helper date function (internal): `getTodayDateString()`

What it does

- Returns **local** date string YYYY-MM-DD (not UTC).

Why it matters

- Ensures prompts align with the day as users see it (e.g., 11/24 local vs 11/25 UTC).
-

7. `getTodayPrompts({ dateString } = {})`

What it does

- Fetches all prompts for a given date (default = today's local date).

Tables touched

- `public.prompts`

Use when

- Building the roulette wheel for today's prompts.
- Showing the list of prompts for a date.

Watch out for

- Requires rows in prompts where `active_on = <date>`.
 - If no prompts exist, returns `prompts: []` and `error: null`. UI should handle empty state.
-

8. `getUserPromptForDate({ dateString } = {})`

What it does

- Gets which prompt the current user has chosen for the given date (default = today).
- Joins prompt info so you get prompt.prompt_text directly.

Tables touched

- public.user_daily_prompts
- public.prompts

Use when

- Showing “Your prompt for today is ...”.
- Pre-filling prompt on the post creation screen.

Watch out for

- If user hasn’t chosen a prompt yet:
 - record will be null, error will be null.
 - Requires user to be logged in.
 - Relies on RLS in user_daily_prompts:
 - SELECT restricted to auth.uid() = user_id.
-

9. setUserPromptForDate({ promptId, dateString, source = "spin" })

What it does

- Sets or changes the current user’s chosen prompt for a given date (default = today).
- Uses UPSERT on the unique (user_id, date) constraint.

Tables touched

- public.user_daily_prompts

Use when

- User confirms a prompt after spinning the wheel.
- User switches to a different prompt from today’s list.

Watch out for

- Requires user to be logged in.
 - RLS:
 - INSERT/UPDATE allowed only when `auth.uid() = user_id`.
 - If there's a unique constraint mismatch (e.g., configuration wrong), you'd see an error with conflict info.
-

Section 3 – Posts (Photo + Caption + Prompt)

Storage bucket

- Bucket: **post-images**
- Path stored in DB: `image_url`, e.g.:
"user-uuid/2025-11-25T04-13-10-289Z.jpg"
- Public URL is created on the frontend with:

```
const { data } = supabase
  .storage
  .from("post-images")
  .getPublicUrl(post.image_url);

const imageUrl = data.publicUrl;
```

10. `uploadPostImage({ file, userId })`

What it does

- Uploads an image file to the post-images bucket.
- Returns the storage **path** (not the full URL).

Tables / Systems touched

- `storage.objects` (via Storage API)
- Bucket: post-images

Use when

- User selects an image to include in a post.

Watch out for

- Requires user logged in (if userId not passed).
 - Storage RLS must allow INSERT into post-images for authenticated users.
 - file must be a File/Blob (web) or equivalent.
-

11. createPost({ promptId, imagePath, caption })

What it does

- Inserts a row into **public.posts** with:
 - user_id = auth.uid()
 - prompt_id = promptId
 - image_url = imagePath
 - caption = caption

Tables touched

- public.posts

Use when

- After successfully uploading an image with uploadPostImage.

Watch out for

- Requires user logged in.
 - promptId must refer to an existing row in prompts.
 - imagePath must be a valid path from uploadPostImage.
 - RLS:
 - INSERT allowed only when auth.uid() = user_id.
-

12. getGlobalFeed({ limit = 50, offset = 0 } = {})

What it does

- Returns newest active posts with:
 - Post data
 - Joined prompt data (prompt.prompt_text)
 - Joined user profile (user.display_name, avatar_url)

Tables touched

- public.posts
- public.prompts
- public.profiles

Use when

- Rendering the global feed screen.
- Scrolling feed (use limit + offset for pagination).

Watch out for

- Only posts with is_active = true are returned.
 - Requires user logged in (because RLS on posts is for authenticated role).
 - For images, you still need to convert image_url to a **public URL** via Storage API as shown above.
-

Section 4 – Reactions (Likes) & Comments

13. likePost({ postId, type = "like" })

What it does

- Creates or updates a reaction row for the current user & post.
- Uses UPSERT on (post_id, user_id) so there's at most one reaction per user per post.

Tables touched

- public.reactions

Use when

- User taps the Like button.

Watch out for

- Requires user logged in.
 - RLS:
 - INSERT allowed when auth.uid() = user_id.
 - Unique constraint on (post_id, user_id) must exist (as designed).
 - If you later add more reaction types, the same helper can still work.
-

14. unlikePost({ postId })

What it does

- Deletes the reaction row for (post_id, auth.uid()).

Tables touched

- public.reactions

Use when

- User taps Like again to “unlike”.

Watch out for

- Requires logged in user.
 - RLS: DELETE only allowed for rows where user_id = auth.uid().
-

15. hasUserLikedPost({ postId })

What it does

- Checks if the current user has a reaction row for the given post.
- Returns:
 - hasLiked boolean
 - reaction row or null

Tables touched

- public.reactions

Use when

- Rendering a post card:
 - If hasLiked → show filled heart.
 - If not → show outline.

Watch out for

- Requires logged in user.
 - RLS: SELECT only for auth.role() = 'authenticated' (we allow reading all reactions, but we filter on user_id client-side).
-

16. getPostLikes({ postId, withUsers = false })

What it does

- Fetches all reactions for a given postId.
- Returns:
 - count = number of reactions (likes)
 - reactions = list of reaction rows
 - If withUsers = true, each reaction includes joined user profile.

Tables touched

- public.reactions
- (optionally) public.profiles via join

Use when

- Displaying number of likes under a post.
- Optionally, showing the list of users who liked the post.

Watch out for

- Returns all reactions regardless of which user did them, but only to authenticated users (RLS).
 - UI should mostly care about count unless you're building a detailed "liked by" view.
-

17. addComment({ postId, body })

What it does

- Inserts a comment row by the current user on the given post.

Tables touched

- public.comments

Use when

- User submits a comment on a post.

Watch out for

- Requires logged in user.
 - body should be non-empty (helper logs error if empty in test page).
 - RLS: INSERT allowed where auth.uid() = user_id.
-

18. `getCommentsForPost({ postId, limit = 50, offset = 0 })`

What it does

- Fetches comments for a given post, newest first, with joined user profile info.

Tables touched

- `public.comments`
- `public.profiles` via `join`

Use when

- Rendering comments section under a post.
- You can paginate with `limit` and `offset`.

Watch out for

- Requires user logged in (SELECT policy is for authenticated).
 - Returns comments: `[]` if none exist; UI should handle that gracefully.
-

19. `deleteComment({ commentId })`

What it does

- Deletes a comment row **only if it belongs to the current user**.

Tables touched

- `public.comments`

Use when

- User deletes their own comment.

Watch out for

- Requires logged-in user.
 - RLS: DELETE only allowed where `auth.uid() = user_id`.
So this helper is safe to call; it won't delete someone else's comment even if you pass their id by mistake—the DB will just reject it.
-

General Gotchas / Patterns to Remember

- **Auth-dependent helpers**
Any helper that uses `supabase.auth.getUser()` will fail if the user is logged out. UI should redirect to login or show a friendly message.

- **Dates (getTodayPrompts, setUserPromptForDate, etc.)**

Use **local time** (our helper already does). Make sure prompts in prompts.active_on match the actual local date you care about.

- **Always check error**

The pattern is always:

```
const { thing, error } = await someHelper(...);
if (error) {
  // show toast / console.error / fallback UI
} else {
  // use `thing`
}
```

? **RLS errors**

If something returns an error like "new row violates row-level security policy", that means:

- The user is logged out, **or**
- The RLS policy for that table/bucket doesn't allow the operation you tried (INSERT/UPDATE/DELETE/SELECT).

These are DB policy issues, not frontend bugs.

? **Storage vs DB**

Uploading an image (uploadPostImage) only creates an object in the Storage bucket. The feed/post logic uses **posts table**, which stores the path. You always need both:

- Storage object (actual file)
- Post row (metadata, prompt, caption, etc.)